README

Helix OTA

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Project root README for Helix OTA — a universal, decoupled over-the-air update system (Go control plane + per-OS client SDKs/agents). First target is Android 15 on Orange Pi 5 Max. The project is currently in its specification/research phase: the authoritative design lives in the spec corpus under `docs/research/main_specs/`, and implementation follows that corpus.
Issues	The spec corpus entry point at `docs/research/main_specs/README.md` is referenced as the corpus index but is not yet present (UNVERIFIED); until it lands, start from the master design (§ links below). The six NEW `ota-*` submodule repositories are not yet created. HelixConstitution clause numbers are carried from the corpus convention and are UNVERIFIED against the authoritative constitution text.
Fixed	N/A (initial revision).
Continuation	Add the corpus index `README.md`; complete the per-component 1.0.0-MVP specs and the open ADRs; create the six PUBLIC `ota-*` repos on GitHub + GitLab; then begin implementation under the corpus.

Table of contents

> Per HelixConstitution §11.4.61 (UNVERIFIED clause number), every document carries the metadata table above followed by this table of contents.

---

1. What is Helix OTA

Helix OTA is a **universal, generic, deeply decoupled** over-the-air (OTA) update system. It comprises a server **control plane**, per-OS **client SDKs/agents**, and a dashboard, designed to be embeddable into any operating system.

- **First target:** Android 15 (all flavors/variants) on the **Orange Pi 5 Max**, where the build pipeline emits flashing images plus an OTA update `.zip` and mandatory hash files.
- **Future targets:** Linux (all flavors), Windows, and other upstream OSes via pluggable OS adapters. These are research/standards-survey only at this stage (`1.X-linux/`, `1.X-windows/`, `1.X-other-os/`).

Operator-stated hard guarantees (from the master design):

- **Zero system corruption** — an update must never brick a working device.
- **Safe upload** — every OTA artifact passes mandatory validation before it can be deployed.
- **Granular rollout** — deploy all at once, or in steps (5%, 10%, 30%, … 100%).
- **Observability** — tracking/measurement/critical-data capture to detect and report problems.
- **Scalability** — single board to millions of devices.

2. Status

Specification / research phase. This repository currently contains the design and research corpus, the documentation export pipeline, and the submodule scaffolding tooling — **not** the implementation. Architecture and stack are locked (see below); component-level decisions that require evidence are deliberately deferred to the open ADRs. Implementation follows the corpus.

No claim of working server, agent, or `ota-*` submodule code is made here; those do not yet exist (anti-bluff, HelixConstitution §7.1 / §11.4.6, UNVERIFIED clause numbers).

3. Mandated technology stack

The stack is locked by operator decision (master design §2 D6, §3):

- **Language / runtime:** Go (control plane, rollout engine, validators); Kotlin/KMP (Android agent).
- **HTTP framework: Gin** (`gin-gonic`).
- **Transport: HTTP/3 (QUIC)** primary with automatic fallback to **HTTP/2** + standard compression; **Brotli** content compression with negotiated **gzip** fallback for older clients.
- **API surface: REST** is the mandated primary surface (`/api/v1`) plus mandatory compatibility interfaces; gRPC is optional/internal only.
- **Persistence:** PostgreSQL (relational), MinIO/S3 (artifact blobs), optional Redis caching only where a measured need exists.
- **Observability:** OpenTelemetry; Prometheus/Grafana surface.

4. Locked architecture

Locked by operator decision (master design §2 D2): **device-side native Android A/B** (`update_engine` + AVB/dm-verity + automatic boot-failure rollback) **plus a custom, decoupled Go control plane**; an OSS engine is wrapped only where it adds value (and that choice is deferred to an evidence-based ADR, D3).

Three planes plus infrastructure, with two deliberately extractable seams — the **OS-adapter** seam (universality) and the **rollout-engine** seam (OS-agnostic campaigns):

- **Control plane** (Go + Gin, HTTP/3→HTTP/2, Brotli): REST API, artifact intake + validation, rollout engine, device registry, telemetry ingest, auth (OAuth2/JWT, RBAC).
- **Data plane:** PostgreSQL, MinIO/S3, OpenTelemetry/Prometheus.
- **Device** (Android 15 on Orange Pi 5 Max): KMP OTA agent (poll/download/verify), AOSP `update_engine` (Virtual A/B), AVB/dm-verity + `boot_control`.
- **Dashboard:** React (login, upload, rollout, fleet health).

Each unit follows the decoupling principle (one purpose, well-defined interface, independently testable; §11.4.28, UNVERIFIED).

5. Specification corpus

The authoritative design and research live under `docs/research/main_specs/`. The intended corpus index is `docs/research/main_specs/README.md` (not yet present — UNVERIFIED). Until it lands, the canonical entry points are:

- **Master design:** `docs/research/main_specs/00-master/2026-06-07-helix-ota-design.md`

- **Submodule reuse map:** docs/research/main_specs/00-master/submodule_reuse_map.md
- **Documentation standards:** docs/research/main_specs/00-master/documentation_standards.md
- **Architecture Decision Records (open):** docs/research/main_specs/research/adr/

Corpus phases: `00-master/`, `research/`, `additions/`, `1.0.0-mvp/`, `1.0.1-staged-rollout/`, `1.X-linux/`, `1.X-windows/`, `1.X-other-os/`.

6. Reusable submodules

Helix OTA is **catalogue-first** (HelixConstitution §11.4.74, UNVERIFIED): every component is satisfied by an existing verified catalogue submodule where one exists; a NEW submodule is created only where the catalogue has no cover. See the submodule reuse map for the full component → submodule bindings.

Six **NEW** reusable, independently-versioned submodules are introduced because no catalogue submodule covers their purpose. Per locked decision D4 they get **PUBLIC** repositories on both GitHub and GitLab. **These repos are not yet created** (UNVERIFIED — URLs below are the planned canonical locations):

Submodule	Purpose (decoupled boundary)	GitHub	GitLab
`ota-protocol`	Shared wire types, manifest schema, status/event enums (Go + KMP); pure contracts, no business logic.	https://github.com/HelixDevelopment/ota-protocol	https://gitlab.com/helixdevelopment1/ota-protocol
`ota-artifact-validator`	Structure/hash/signature/metadata validation pipeline; OS-aware via plugins, no transport.	https://github.com/HelixDevelopment/ota-artifact-validator	https://gitlab.com/helixdevelopment1/ota-artifact-validator
`ota-rollout-engine`	Staged-rollout + halt/advance logic, OS-agnostic; no HTTP, pure engine + storage port.	https://github.com/HelixDevelopment/ota-rollout-engine	https://gitlab.com/helixdevelopment1/ota-rollout-engine
`ota-update-engine-bridge`	Wrapper over AOSP `update_engine` / `boot_control`;	https://github.com/HelixDevelopment/ota-update-engine-bridge	https://gitlab.com/helixdevelopment1/ota-update-engine-bridge

Submodule	Purpose (decoupled boundary)	GitHub	GitLab
	Android-only, thin, testable.		
`ota-android-agent`	KMP device agent (poll/ download/verify/ apply/report); consumes protocol + bridge.	https://github.com/HelixDevelopment/ota-android-agent	https://gitlab.com/helixdevelopment1/ota-android-agent
`ota-telemetry-schema`	Telemetry event/ metric schema + codecs; shared by server + agents, no transport/ storage.	https://github.com/HelixDevelopment/ota-telemetry-schema	https://gitlab.com/helixdevelopment1/ota-telemetry-schema

Reused catalogue submodules (verified canonical names from documentation standards §9) include `auth`, `security`, `database`, `Storage`, `observability`, `eventbus`, `ratelimiter`, `middleware`, `http3`, `mdns`, `recovery`, `Herald`, `config`, `discovery`, `cache`, `docs_chain` / `Document` / `Formatters`, `containers`, and the KMP set `Auth-KMP`, `Security-KMP`, `Storage-KMP`, `Config-KMP`. New repos can be bootstrapped with `scripts/scaffold_submodule.sh`, which scaffolds the README (metadata table), Apache-2.0 license, `.gitignore`, `docs/` and `tests/` placeholders, then configures and pushes the GitHub + GitLab remotes.

7. Governance — HelixConstitution

Helix OTA is governed by the **HelixConstitution**, the standards the corpus is authored against. Key rules applied throughout (clause numbers carried from the corpus convention and **UNVERIFIED** against the authoritative constitution text):

- **Four-layer testing + mutation meta-test** (§1) — source-presence gate → artifact gate → runtime/integration → mutation meta-test, with no-bluff positive evidence.
- **Anti-bluff** (§7.1 / §11.4.6) — no fabricated facts; anything unconfirmed is marked `UNVERIFIED`. This README follows that rule.
- **Catalogue-first reuse** (§11.4.74) and the **decoupling principle** (§11.4.28).
- **Containers substrate** (§11.4.76).
- **Metadata table + table of contents on every document** (§11.4.61).
- **Subagent-driven authoring with a mandatory code-review gate** (§11.4.20 / §11.4.125).
- **Multi-upstream mirroring** (§2 / §2.1) — every repo is pushed to GitHub and GitLab.

8. Documentation export pipeline

Canonical source is **Markdown + Mermaid**. The multi-format export pipeline `scripts/export_docs.sh` converts the corpus into HTML and DOCX (plus PDF when a LaTeX/PDF engine is available) and renders embedded Mermaid diagrams to SVG and PNG, writing outputs into per-directory `_exports/` folders.

It **degrades gracefully and honestly**: each renderer (pandoc, mermaid-cli, a LaTeX/PDF engine) is probed at runtime, missing tools are logged as `SKIPPED` rather than failing the run, and a format is only reported as produced if its file actually exists (export pipeline notes, §11.4.65 / §7.1, UNVERIFIED clause numbers).

```
# Export the whole corpus (default target) or a single file/dir:
scripts/export_docs.sh
scripts/export_docs.sh docs/research/main_specs/00-master/
        2026-06-07-helix-ota-design.md
```

9. Repository layout

```
.
├── README.md                  this file
├── LICENSE                    Apache-2.0
├── docs/
│   ├── request/               operator request inputs
│   └── research/main_specs/  authoritative design + research
corpus (see §5)
├── scripts/
│   ├── export_docs.sh         multi-format documentation export
pipeline (§8)
│   └── scaffold_submodule.sh scaffold + push a new reusable
submodule (§6)
└── upstreams/                 multi-upstream remote host
definitions (GitHub, GitLab, …)
```

10. Tracked-Items + Status Documents

Doc-map for the canonical tracker documents plus the key spec/status documents in the active spec corpus (`docs/research/main_specs/`). Each row links the canonical Markdown source plus its HTML/PDF exports where those siblings exist on disk; `Revision` and `Last modified` are read from each document's own metadata header (– = absent). Every linked path was verified to exist on disk (HelixConstitution §11.4.57 / §7.1 anti-bluff, UNVERIFIED clause numbers).

Document	Last modified	Revision	Markdown	HTML	PDF	DOCX
Issues — open workable items	2026-06-19T08:30:00Z	3	md	html	pdf	—
Issues Summary — open, short-form	2026-06-10T18:30:00Z	2	md	html	pdf	—

Document	Last modified	Revision	Markdown	HTML	PDF	DOCX
Fixed — closed workable items	2026-06-10T18:30:00Z	2	md	html	pdf	—
Fixed Summary — closed, short-form	2026-06-10T18:30:00Z	2	md	html	pdf	—
Feature Inventory — Status (§11.4.153 four-format)	2026-06-19T13:30:00Z	4	md	html	pdf	docx
Feature Inventory — Status Summary	2026-06-19T13:30:00Z	4	md	html	pdf	docx
Emulator (rk3588_ab_virt) — Status	2026-06-11T13:15:00Z	3	md	html	pdf	—
Emulator (rk3588_ab_virt) — Status Summary	2026-06-11T13:15:00Z	3	md	html	pdf	—
Spec-corpus continuation / handoff	2026-06-10	3	md	html	pdf	—
Additions synthesis (gap closure)	2026-06-08	3	md	html	pdf	—
1.0.0-MVP API — operational endpoints	2026-06-08	1	md	html	pdf	—
1.0.0-MVP API — implemented endpoints	2026-06-08T00:00:00Z	1	md	html	pdf	—
1.0.0-MVP dashboard design	2026-06-08	1	md	html	pdf	—
1.0.1 staged rollout — overview (README)	2026-06-08	2	md	html	pdf	—
1.0.1 staged rollout — rollout engine	2026-06-08	1	md	html	pdf	—
1.0.1 staged rollout — migration 002 design	2026-06-08	1	md	html	pdf	—
	2026-06-08	1	md	html	pdf	—

Document	Last modified	Revision	Markdown	HTML	PDF	DOCX
1.0.1 staged rollout — device TUF						
1.0.1 staged rollout — rollback UX	2026-06-08	1	md	html	pdf	—
1.0.3 delta updates — overview (README)	2026-06-08	1	md	html	pdf	—
1.0.3 delta updates — design	2026-06-08	1	md	html	pdf	—
Repo public-visibility audit (gap G11)	2026-06-08T00:00:00Z	1	md	html	pdf	—

11. License

Apache-2.0. See `LICENSE`.